

# Software Requirements Specification

for

## Smart Parking Management System

**Version 1.0**

Prepared by Amit Kumar, Ankit Modi, Arpit Gupta, Anubhav Mittal

Course: CS311L — Software Engineering and Design Principles

Institution: KIET Deemed to be University

Project Type: Software-only web-based application

Document Status: Academic Project SRS

Date: 17 September 2026

*Scope note: This is a software-only Smart Parking Management System. Physical parking sensors, RFID readers, cameras, barriers, and other hardware are outside the current project scope.*

## Table of Contents

1. Introduction — 1.1 Purpose · 1.2 Document Conventions · 1.3 Intended Audience and Reading Suggestions · 1.4 Product Scope · 1.5 References · 1.6 Requirement Elicitation

2. Overall Description — 2.1 Product Perspective · 2.2 Product Functions · 2.3 User Classes and Characteristics · 2.4 Operating Environment · 2.5 Design and Implementation Constraints · 2.6 User Documentation · 2.7 Assumptions and Dependencies

3. External Interface Requirements — 3.1 User Interfaces · 3.2 Hardware Interfaces · 3.3 Software Interfaces · 3.4 Communications Interfaces

4. System Features — 4.1 User Registration and Authentication · 4.2 Parking Location and Slot Management · 4.3 Parking Availability and Search · 4.4 Slot Reservation and Cancellation · 4.5 Vehicle Entry and Exit Records · 4.6 Parking Fee and Payment Records · 4.7 Notifications and User History · 4.8 Admin Dashboard and Reports

5. Other Nonfunctional Requirements — 5.1 Performance · 5.2 Safety · 5.3 Security · 5.4 Software Quality Attributes · 5.5 Business Rules (Decision Table)

6. Other Requirements — 6.1 Database · 6.2 Backup and Recovery · 6.3 Internationalization and Accessibility · 6.4 Future Expansion

Appendix A: Glossary · Appendix B: Analysis Models · Appendix C: To Be Determined List

## Revision History

| Name                                                | Date           | Reason For Changes                                                                            | Version |
|-----------------------------------------------------|----------------|-----------------------------------------------------------------------------------------------|---------|
| Anubhav Mittal                                      | September 2026 | Initial SRS for academic project                                                              | 1.0     |
| Amit Kumar, Ankit Modi, Arpit Gupta, Anubhav Mittal | September 2026 | Reformatted to team IEEE template; added elicitation section and business-rule decision table | 1.1     |

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Smart Parking Management System, Version 1.0. The system is intended to provide a centralized software platform through which users can view parking locations, check slot availability, reserve a slot, manage vehicle details, maintain entry/exit records, view parking charges, and review parking history. Administrators can manage parking locations, slots, reservations, users, pricing information, and reports.

The SRS provides a common requirements baseline for stakeholders, developers, designers, testers, and academic evaluators. It also provides the basis for the requirement report, UML models, architecture, implementation, and testing activities.

## 1.2 Document Conventions

The document follows the section organization of the provided IEEE-style SRS reference template. Functional requirements use FR-xx, non-functional requirements use NFR-xx, and business rules use BR-xx. Priorities are High, Medium, or Low. Requirement statements are intended to be concise, unambiguous, necessary, verifiable, and testable.

## 1.3 Intended Audience and Reading Suggestions

The intended audience includes the project team, faculty/guide, developers, UI/UX designers, testers, system administrators, and academic evaluators. Readers should review Sections 1–2 for context, Section 3 for interfaces, Section 4 for detailed functionality, and Section 5 for quality and operational constraints.

## 1.4 Product Scope

The Smart Parking Management System is a web-based software application that digitizes common parking management activities. It helps users locate suitable parking, check software-recorded availability, reserve slots, manage vehicles, and review parking activity. Administrators receive centralized control over parking resources.

The current academic scope is software-only. Availability is represented using application data and administrator/session updates rather than physical sensor input. Hardware integrations may be future extensions but are not required for Version 1.0.

## 1.5 References

- CS311L Software Engineering and Design Principles syllabus provided for the course.
- IEEE-style SRS template provided for the project.
- Project-specific requirements defined in this SRS.

## 1.6 Requirement Elicitation

Requirements for this SRS were gathered using the following techniques:

- Course syllabus and faculty-guide consultation to align scope with CS311L academic expectations.
- Study of existing parking-management and reservation applications to benchmark common features.
- Team brainstorming sessions among Amit Kumar, Ankit Modi, Arpit Gupta, and Anubhav Mittal to walk through the driver and administrator journeys end to end.
- Use-case walkthroughs to validate that each identified function traced back to a real user need.
- Iterative peer review of draft requirements to check for ambiguity, testability, and completeness before finalizing FR/NFR/BR identifiers.

## 2. Overall Description

### 2.1 Product Perspective

The product is a new, self-contained software system for managing parking operations. A typical deployment consists of a browser-based client, an application/backend server, and a database. Users interact through the web interface while administrators use role-protected management screens.

High-level logical view: User / Administrator → Web Interface → Application Server → Database. Optional email/SMS or online payment services may be integrated later and are not mandatory dependencies for Version 1.0.

### 2.2 Product Functions

- User registration, login, logout, and profile management.
- Vehicle registration and management.
- Parking location creation and slot configuration by administrators.
- Display of slot availability based on application records.
- Search and filtering of parking locations.
- Slot reservation, confirmation, and cancellation.
- Vehicle entry and exit records.
- Parking charge calculation and payment/charge status records.
- Notifications and reservation/parking history.
- Administrative dashboard, reports, and utilization summaries.

### 2.3 User Classes and Characteristics

| User Class            | Characteristics / Responsibilities                                                                                |
|-----------------------|-------------------------------------------------------------------------------------------------------------------|
| Parking User          | Registers, manages vehicles, searches parking, views availability, makes/cancels reservations, and views history. |
| Parking Administrator | Manages locations, slots, pricing, reservations, occupancy/session records, and reports.                          |
| System Administrator  | Manages system-level access, administrator accounts, configuration, and maintenance controls.                     |

### 2.4 Operating Environment

- Modern web browser such as Chrome, Edge, or Firefox.
- Desktop, laptop, tablet, or smartphone.
- Application server capable of hosting the selected backend.
- Database server supported by the selected stack.
- Windows/Linux or another supported server/development OS.
- Internet or local-network connectivity.

### 2.5 Design and Implementation Constraints

- Version 1.0 shall be software-only and shall not require physical parking sensors or gate hardware.
- The stack must support web-based client-server operation and persistent storage.
- Administrative operations must be protected by authentication and authorization.
- Requirements, UML models, architecture, implementation, and tests should remain consistent.
- Production-grade payment settlement and physical access control are outside the initial scope.

## 2.6 User Documentation

The project should provide a short user guide or README covering account creation, login, vehicle management, parking search, reservation, cancellation, history, and administrator operations. The GitHub repository should include setup and execution instructions.

## 2.7 Assumptions and Dependencies

- Users have browser and network access.
- Administrators maintain correct parking and slot data.
- Optional notification services depend on their external providers.
- Optional payment integration depends on a compatible payment provider.
- Future hardware integrations are not assumed by Version 1.0.

# 3. External Interface Requirements

## 3.1 User Interfaces

- The system shall provide a responsive web interface for desktop and mobile-sized screens.
- User screens shall include Home/Dashboard, Registration, Login, Profile, Vehicle Management, Parking Search, Parking Details, Reservation, Charge Summary, and History.
- Administrative screens shall include Admin Dashboard, User Management, Parking Location Management, Slot Management, Pricing, Reservations, and Reports.
- Forms shall identify required fields and show validation/error messages.
- Destructive actions such as cancellation/deletion should use confirmation dialogs.
- Availability shall use clear labels such as Available, Reserved, Occupied, or Unavailable.

## 3.2 Hardware Interfaces

No dedicated hardware interface is required for Version 1.0. The system is accessed through standard computing devices. Physical sensors, RFID readers, cameras, automated barriers, and IoT devices are explicitly outside the current project scope.

## 3.3 Software Interfaces

- Frontend communicates with the backend through HTTP/HTTPS APIs.
- Backend communicates with the selected database through its driver/ORM.
- Authentication may use session-based or token-based mechanisms.
- Optional notification/payment services may be integrated through documented APIs.
- The exact framework/database versions are implementation decisions to be recorded in the project documentation.

## 3.4 Communications Interfaces

- Client-server communication shall use HTTP/HTTPS.
- HTTPS should be used in deployed environments.
- API requests/responses should use a structured format such as JSON.
- Authentication credentials/tokens shall not be transmitted through insecure mechanisms.
- Communication failures shall produce clear messages without exposing sensitive server information.

## 4. System Features

Functional requirements are organized by major system services, following the feature-oriented structure of the supplied SRS template. Each feature's stimulus/response sequence follows the same general pattern: user/administrator initiates an action → system validates input and current data → system performs the operation → persistent records are updated → system returns confirmation or an appropriate error.

### 4.1 User Registration and Authentication

#### 4.1.1 Description and Priority

Priority: High. Provides account creation and access control.

#### 4.1.2 Functional Requirements

- FR-01:** The system shall allow a parking user to create an account using required registration information.
- FR-02:** The system shall validate required registration fields before creating an account.
- FR-03:** The system shall prevent registration with an already registered email address.
- FR-04:** The system shall authenticate registered users using valid credentials.
- FR-05:** The system shall reject invalid credentials with an appropriate error message.
- FR-06:** The system shall allow an authenticated user to log out.
- FR-07:** The system shall enforce role-based access for administrative functions.

### 4.2 Parking Location and Slot Management

#### 4.2.1 Description and Priority

Priority: High. Allows administrators to maintain parking locations, slots, and pricing.

#### 4.2.2 Functional Requirements

- FR-08:** The system shall allow an authorized administrator to create a parking location with name, address, description, and operating information.
- FR-09:** The system shall allow an administrator to add, update, activate, or deactivate parking slots.
- FR-10:** Each parking slot shall have a unique identifier within its parking location.
- FR-11:** The system shall maintain slot status as Available, Reserved, Occupied, or Unavailable.
- FR-12:** The system shall prevent an inactive slot from being offered for new reservations.
- FR-13:** The system shall allow administrators to configure parking charges or pricing rules.
- FR-14:** Deactivating a slot/location shall not break existing reservation or history records.

### 4.3 Parking Availability and Search

#### 4.3.1 Description and Priority

Priority: High. Enables users to discover parking locations and view application-recorded availability.

#### 4.3.2 Functional Requirements

- FR-15:** The system shall allow users to search parking locations using location-related criteria.
- FR-16:** The system shall display matching parking locations.
- FR-17:** The system shall display slot counts by status for a selected location.
- FR-18:** The system shall allow supported filters such as availability and price range.
- FR-19:** The system shall display relevant parking details before reservation.
- FR-20:** The system shall refresh availability when the user requests the latest status.

**FR-21:** Inactive parking locations shall not be offered for new reservations.

**FR-22:** The system shall display a suitable message when no matching location is found.

## 4.4 Slot Reservation and Cancellation

### 4.4.1 Description and Priority

Priority: High. Allows authenticated users to reserve and manage eligible parking slots.

### 4.4.2 Functional Requirements

**FR-23:** The system shall allow an authenticated user to select an available slot and provide required reservation details.

**FR-24:** The system shall validate that the selected slot is still available before confirmation.

**FR-25:** The system shall generate a unique reservation identifier.

**FR-26:** A successfully reserved slot shall change to Reserved.

**FR-27:** The system shall display reservation details including location, slot, vehicle, time, and charge.

**FR-28:** The system shall allow cancellation when permitted by business rules.

**FR-29:** A cancelled slot shall return to Available when applicable.

**FR-30:** The system shall prevent overlapping active reservations for the same slot.

## 4.5 Vehicle Entry and Exit Records

### 4.5.1 Description and Priority

Priority: Medium. Maintains software records of actual parking sessions.

### 4.5.2 Functional Requirements

**FR-31:** Authorized staff/administrators shall be able to record vehicle entry against an eligible reservation.

**FR-32:** The system shall record entry date and time.

**FR-33:** Authorized staff/administrators shall be able to record vehicle exit.

**FR-34:** The system shall record exit date and time.

**FR-35:** The system shall update slot status according to the configured entry/exit workflow.

**FR-36:** The system shall prevent an exit record when no active parking session exists.

**FR-37:** Authorized users shall be able to view parking session records.

## 4.6 Parking Fee and Payment Records

### 4.6.1 Description and Priority

Priority: High. Calculates applicable charges and maintains payment-related status.

### 4.6.2 Functional Requirements

**FR-38:** The system shall calculate parking charges according to the configured pricing rule.

**FR-39:** The system shall display estimated or final charges.

**FR-40:** The system shall maintain a charge/payment record linked to the reservation or session.

**FR-41:** The system shall maintain a payment status such as Pending, Paid, Failed, or Refunded when applicable.

**FR-42:** The system shall display an appropriate message when a charge/payment operation fails.

**FR-43:** Version 1.0 shall not require physical cash handling or hardware-based payment collection.

## 4.7 Notifications and User History

### 4.7.1 Description and Priority

Priority: Medium. Keeps users informed and provides historical activity.

### 4.7.2 Functional Requirements

- FR-44:** The system shall display an in-application confirmation after successful reservation.
- FR-45:** The system shall notify the user when a reservation is created or cancelled.
- FR-46:** The system shall provide reservation and parking-session history.
- FR-47:** The system shall allow users to view current and past reservation status.
- FR-48:** The system shall display relevant error/status messages when an operation cannot be completed.

## 4.8 Admin Dashboard and Reports

### 4.8.1 Description and Priority

Priority: High. Provides centralized operational monitoring.

### 4.8.2 Functional Requirements

- FR-49:** The system shall provide an administrator dashboard with summary information about locations, slots, users, and reservations.
- FR-50:** Authorized administrators shall be able to view and manage registered users.
- FR-51:** Authorized administrators shall be able to view and manage reservations.
- FR-52:** The system shall provide parking utilization information from stored records.
- FR-53:** The system shall provide reports or summaries for reservations, sessions, charges, and slot utilization.
- FR-54:** Administrative reports and management functions shall be restricted to authorized roles.

## 5. Other Nonfunctional Requirements

### 5.1 Performance Requirements

- NFR-01:** Common pages should become usable within approximately 3 seconds under normal project load.
- NFR-02:** Standard parking search and availability requests should return results within approximately 3 seconds.
- NFR-03:** The initial design target is at least 100 concurrent active sessions, subject to deployment resources.
- NFR-04:** Dashboard queries should use efficient database operations to remain responsive.
- NFR-05:** Reservation creation should use suitable validation/transaction controls to avoid duplicate bookings.

### 5.2 Safety Requirements

- NFR-06:** Version 1.0 shall not control physical gates, barriers, or vehicles.
- NFR-07:** Software-recorded availability shall not be presented as a guarantee of physical availability where no sensor verification exists.
- NFR-08:** Destructive administrative actions should require confirmation.
- NFR-09:** Errors shall be handled without exposing internal stack traces to normal users.

### 5.3 Security Requirements

- NFR-10:** Passwords shall be stored using secure password hashing, not plain text.

**NFR-11:** Authenticated sessions shall be protected against unauthorized access.

**NFR-12:** Administrative APIs/pages shall enforce server-side authorization.

**NFR-13:** User input shall be validated and sanitized against malformed or malicious input.

**NFR-14:** HTTPS shall be used in deployed environments for credential and sensitive-data transmission.

**NFR-15:** Passwords, tokens, and secrets shall not be exposed in normal UI messages or logs.

**NFR-16:** Users shall access only data and operations permitted by their role.

**NFR-17:** Database credentials and API secrets shall be stored outside source-controlled code.

**NFR-18:** Important administrative and operational actions should be logged for accountability and troubleshooting.

## 5.4 Software Quality Attributes

**NFR-19 (Usability):** Common search and reservation tasks should be understandable without technical knowledge.

**NFR-20 (Reliability):** Failed requests should be controlled and should not corrupt reservation records.

**NFR-21 (Maintainability):** The system should be divided into logical modules such as authentication, parking, reservation, vehicle, charges, and administration.

**NFR-22 (Testability):** Modules/APIs should support unit and integration testing.

**NFR-23 (Portability):** The web application should work across major modern browsers.

**NFR-24 (Scalability):** The design should support growth in locations, slots, users, and administrators.

**NFR-25 (Availability):** The application should remain accessible during normal deployment operating periods.

**NFR-26 (Correctness):** Slot state, reservation state, and charge calculations shall use consistent stored data.

**NFR-27 (Reusability):** Common validation, authentication, and data-access functionality should be reusable.

**NFR-28 (Interoperability):** APIs should use documented standard web formats for future integration.

## 5.5 Business Rules — Decision Table

Business rules are expressed as a condition/decision table: given the stated condition, the system applies the corresponding rule, cross-referenced to the functional requirement(s) that enforce it.

| BR ID | Condition                                                                  | Rule / Decision                                                           | Enforced By        |
|-------|----------------------------------------------------------------------------|---------------------------------------------------------------------------|--------------------|
| BR-01 | A user attempts to create or manage a reservation                          | Only authenticated users may proceed; unauthenticated attempts are denied | FR-04, FR-23       |
| BR-02 | A slot already has an active reservation overlapping the requested time    | The new overlapping reservation request is rejected                       | FR-30              |
| BR-03 | The requested slot's status is Reserved, Occupied, or Unavailable          | The reservation request is rejected; only Available slots may be reserved | FR-24              |
| BR-04 | The location or slot is marked inactive                                    | The location/slot is not offered for new reservations                     | FR-12, FR-21       |
| BR-05 | A cancellation is requested outside the configured cancellation conditions | The cancellation request is rejected                                      | FR-28              |
| BR-06 | A reservation is successfully confirmed                                    | The system generates and assigns a unique reservation identifier          | FR-25              |
| BR-07 | A vehicle record is created                                                | The vehicle record must be linked to an existing user account             | Vehicle management |

|       |                                                          |                                                                           |                          |
|-------|----------------------------------------------------------|---------------------------------------------------------------------------|--------------------------|
| BR-08 | A user attempts an administrative operation              | Access is granted only if the user's role is Administrator                | FR-07, FR-54             |
| BR-09 | A parking charge is calculated for a reservation/session | The pricing rule of the selected location/session is applied              | FR-38                    |
| BR-10 | A parking session is marked complete                     | The session record must include an entry time, and an exit time if closed | FR-32, FR-34             |
| BR-11 | A reservation or session state changes                   | Slot status must be updated to remain consistent with that state          | FR-11, FR-35             |
| BR-12 | No physical hardware sensor is present at the facility   | Sensor data is not required as a source of truth in Version 1.0           | Scope note (Section 1.4) |

## 6. Other Requirements

### 6.1 Database Requirements

- The database shall persist users, vehicles, parking locations, slots, reservations, parking sessions, charges/payment records, and relevant administrative data.
- Major entities shall have unique identifiers and defined relationships.
- Historical reservation/session/charge records shall remain referentially consistent.

### 6.2 Backup and Recovery

- The deployment should support periodic database backups.
- Backup and restoration procedures should be documented.
- Recovery should restore the latest valid backup without changing application requirements.

### 6.3 Internationalization and Accessibility

- The UI should use clear labels, readable text, consistent navigation, and accessible form controls.
- The design should not rely only on color to communicate slot status.
- The architecture should permit future multilingual support.

### 6.4 Future Expansion

Potential future extensions include IoT sensor integration, QR/Rfid check-in, automated gates, ANPR/camera integration, online payment gateways, dynamic pricing, map/navigation integration, and predictive parking analytics. These are outside Version 1.0.

## Appendix A: Glossary

| Term                  | Definition                                               |
|-----------------------|----------------------------------------------------------|
| SRS                   | Software Requirements Specification.                     |
| Parking User          | Registered user who searches and reserves parking.       |
| Parking Administrator | Authorized user managing parking operations.             |
| Parking Slot          | Individual parking position represented by the software. |
| Reservation           | Confirmed request to use an eligible parking slot.       |
| Parking Session       | Recorded period associated with vehicle entry and exit.  |
| FR                    | Functional Requirement.                                  |
| NFR                   | Non-Functional Requirement.                              |
| BR                    | Business Rule.                                           |
| API                   | Application Programming Interface.                       |

## Appendix B: Analysis Models

The following models are to be prepared from this SRS for the course assessment: UML Use Case Diagram and narratives, UML Class Diagram, Sequence Diagrams, Activity Diagram, System Architecture Diagram, Data Flow Diagram, and an ER/Data Model. The models should remain traceable to the FR/NFR requirements.

## Appendix C: To Be Determined List

| TBD ID | Item                        | Status / Decision Point                        |
|--------|-----------------------------|------------------------------------------------|
| TBD-01 | Frontend framework/version  | To be selected during implementation planning. |
| TBD-02 | Backend framework/version   | To be selected during implementation planning. |
| TBD-03 | Database technology/version | To be selected during implementation planning. |
| TBD-04 | Payment gateway             | Optional; not required for core Version 1.0.   |
| TBD-05 | Email/SMS provider          | Optional; not a core dependency.               |
| TBD-06 | Deployment platform         | To be selected before deployment/demo.         |